Bi-word index:

A biword index is an information retrieval technique that **indexes consecutive pairs of words (biwords) as single vocabulary terms**, facilitating efficient phrase searches. By storing pairs like "stanford university" and "palo alto," it enables **fast retrieval of 2-word phrase queries**. However, it can cause index explosion and often requires combination with single-word indexes to be effective.

Biword indexes only guarantee **adjacent word pairs**, not the **full phrase sequence**.

What False Positive Problem?

In a **biword index**, a **false positive** occurs when the system returns a document as a match for a phrase query—even though the **exact phrase does not actually appear** in that document.

Example:

Document d: *"new york is a big city"*

So, bi-word index should contain:

- new york
- york is
- is a
- a big
- big city

What the result of this query: "new york city"?

Its biwords will be → "new york" and "york city"

While the bi-word index doesn't contain "york city", so **document d will not be returned.**

Note: If preprocessing or indexing is imperfect (e.g., ignoring stopwords or punctuation), the system might still:

- Match *new york*
- Match *york city* indirectly or via normalization

  ❖ The document could be retrieved **incorrectly** → **False Positive**

Exercise 2.8 on bi-word index

Assume a **biword index.** Give an example of a document which will be returned for a query of: "New York University" but is actually a false positive which should not be returned.

Exercise 2.8 Answer

A biword index is a type of index that treats consecutive words as pairs. For example, in the query "New York University", the biword index would be ["New York", "York University"].

A document that could be returned as a **false positive** for this query could be one that contains the sentence:

"He moved from New York to a University in California."

In this case, the biwords "New York" and "York University" are present, but they do not refer to the institution "New York University".

Summary of Web Crawling vs. Web Scraping

The following table summarizes the main differences between the two concepts:

	Web Crawling	Web Scraping
Purpose	Finding new web pages	Extracting specific data from web pages
Performed by	Automated crawling bots	Automated scraping scripts
Process	Following links to build a map of URLs	Parsing HTML content to extract data using specific instructions
Result	A list of web pages	Data in JSON, CSV, or similar formats

Is Web Scraping the Same as Web Crawling?
No, web scraping isn't the same as web crawling. The **latter** is about exploring the web to discover new pages, while **web scraping** involves retrieving data from HTML documents. Scraping generally involves and requires web crawling, thus web crawling is a subset of web scraping

Web crawling gathers pages to create indices or collections.

On the other hand, **web scraping** downloads pages to extract a specific set of data for analysis purposes, for example, product details, pricing information, SEO data, or any other data sets.

Web crawler scans and indexes whereas web scraper scans, finds and returns the relevant information.

1. Web Scraping :
Web Scraping is a technique used to extract a large amount of data from websites and then saving it to the local machine in the form of XML, excel or SQL. The tools used for web scraping are known as **web scrapers**. On the basis of the requirements given, they can extract the data from any website in a fraction of time. This automation of tasks is very helpful for developing data for machine learning and other purpose. They work in four steps:
1. Sending the request to the target page.
2. Getting response from the target page.
3. Parsing and extracting the response.
4. Download the data.

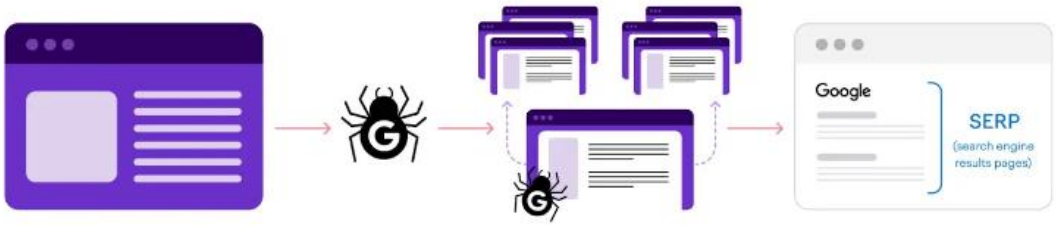
Some of the popular web scraping tools are ProWebScraper, Webscraper.io, etc.

2. Web Crawling :
Web Crawling is analogous to a spider crawling but the place of crawling here is the web!. It basically visits a website and read web pages for the purpose of building entries for search engine index. The tools that are used for web crawling are known as **web crawlers** or **spiders**. A series of web pages are analyzed and links to the pages on them are then followed for even more links thus it does a deep search for extracting of information. Famous search engines such as Google, Yahoo and Bing do web crawling and use this information for indexing web pages. Examples are Scrapy and Apache nut.

Difference between Web Scraping and Web Crawling :

S.NO.	Web Scraping	Web Crawling
1.	The tool used is **Web Scraper**.	The tool used **Web Crawler** or **Spiders**.
2.	It is used for **downloading** information	It is used for **indexing** of Web pages
3.	It **need not visit all the pages** of website for information.	It **visits each and every page**, until the last line for information.
4.	It is done on both **small and large scale**.	It is mostly employed in **large scale**.
5.	Application areas include Retail Marketing, Equity search, and Machine learning.	Used in search engines to give search results to the user.
6.	This **needs crawl agent and a parser** for parsing the response.	This only **needs only crawl agent**.
7.	ProWebScraper, Web Scraper.io are the examples	Google, Yahoo or Bing do Web Crawling

How Search Engines Work



You publish content on your site

Google bots or "spiders" crawl your site and review pages

Google indexes your page

If your page meets the ranking criteria, Google will show it in its search results

These are the following steps to create a web crawler:

1. In the first step, we first pick a URL from the specified URL list.
2. Fetch the HTML code of that URL.
3. Get the links to the other URLs by parsing the HTML code.
4. Check whether the URL is already crawled before or not. We also check whether we have seen the same content before or not. If both conditions don't match, we add them to the **index**.
5. For each extracted URL, verify that whether they agree to be checked (robots.txt, crawling frequency)

Web crawler algorithm

```
public void getPageLinks(String URL) {
    //4. Check if you have already crawled the URLs
    //(we are intentionally not checking for duplicate content in this example)
    if (!links.contains(URL)) {
        try {
            //4. (i) If not add it to the index
            if (links.add(URL)) {
                System.out.println(URL);
            }

            //2. Fetch the HTML code
            Document document = Jsoup.connect(URL).get();//jsoup jar to extract web data
            //3. Parse the HTML to extract links to other URLs
            Elements linksOnPage = document.select("a[href]");

            //5. For each extracted URL... go back to Step 4.
            for (Element page : linksOnPage) {
                getPageLinks(page.attr("abs:href"));
            }
        } catch (IOException e) {
            System.err.println("For '" + URL + "': " + e.getMessage());
        }
    }
}
```

https://subscription.packtpub.com/book/big-data-and-business-intelligence/9781787122536/1/ch01lvl1sec20/extracting-web-data-from-a-url-using-jsoup

(Data Scraping examples)
https://youtu.be/e2ndsH8g8BQ?si=zVTE1SWGwGh8P5nV
https://youtu.be/ct7IDj-E58M?si=-rqKry0b1C3pO2jG

(Difference between web crawling & web scraping)
https://youtu.be/NHHdGC0qvIQ?si=9aDsEEOnEqVHRa-N

(web scraping using Java)
https://youtu.be/BEvRZUEQ3Dc?si=FjtBkz1dLkmkXEpH